Algoritmos de ordenamientos, repaso

Programación I

22

Octubre

UNRN

Universidad Nacional de **Río Negro**

r27

Programación I

Algoritmos de ordenamientos

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

Prof. Daniel Teira/Diego Diaz

Sede Bariloche

Contenidos de la clase

- **Repaso de algoritmos: quicksort iterativo**
- **Ejemplos & Repaso**

Quicksort iterativo

En lugar de que la función se llame a sí misma para procesar los subarreglos, guardas los índices de inicio (low) y fin (high) en un arreglo temporal. El bucle while continuará sacando pares de índices de esta pila, particionando el arreglo, y metiendo los nuevos índices de los subarreglos resultantes hasta que la pila quede completamente vacía.

Motivo	Descripción
Evitar Stack Overflow	La recursividad utiliza la memoria de la pila del sistema (Stack), que es muy limitada (usualmente unos pocos megabytes). Si intentas ordenar millones de registros, la versión recursiva clásica colapsará el programa.
Control de Memoria	Al usar malloc y gestionar tu propia pila en el Heap (memoria dinámica), puedes usar casi toda la memoria RAM disponible en tu computadora, permitiendo ordenar volúmenes de datos masivos sin que el programa aborte (crash).
Sistemas Embebidos	En microcontroladores o sistemas con recursos críticos donde no se permite la recursión por políticas de seguridad o hardware estricto, la iteración es obligatoria.

Repaso de algoritmos vistos

- **Bubble Sort**
- **Selection Sort**
- **Insertion Sort**

→ Complejidad N2

Tiempo Promedio y Peor Caso	$O(N^2)$. Si duplicas la cantidad de datos, el tiempo de ejecución **se cuadruplica**.
Cuándo son útiles	Útiles solo para listas muy pequeñas (menos de 50-100 elementos) o, en el caso de *Insertion Sort*, si sabes de antemano que la lista ya está casi totalmente ordenada.
Impacto real	Si ordenar 1.000 elementos tarda 1 segundo, ordenar 10.000 elementos (10 veces más) tardará unos **100 segundos**.
Memoria Adicional	$O(1)$. Su gran ventaja es que ordenan *in-place*; no necesitan arreglos extra, solo un par de variables temporales.

- **Quicksort**
- **Mergesort**

→ Complejidad NlogN

Tiempo Promedio	$O(N \log N)$. Si duplicas los datos, el tiempo solo aumenta un poco más del doble.
Cuándo son útiles	Son los algoritmos estándar de la industria. Lenguajes como C, Java o Python utilizan variaciones de estos debajo del capó para sus funciones de ordenamiento nativas (`qsort` , `sort()`).
Impacto real	Si ordenar 1.000 elementos tarda 1 segundo, ordenar 10.000 elementos (10 veces más) tardará apenas **13 o 14 segundos**. Para listas de un millón de registros, $O(N^2)$ podría tardar días, mientras $O(N \log N)$ tarda fracciones de segundo.

Quicksort iterativo

La decisión de usar Quick Sort iterativo en lugar de la versión recursiva clásica se reduce a un factor crítico: **el control absoluto de la memoria para evitar que el programa colapse** (el famoso *Stack Overflow*).

- Evitar el "Stack Overflow" (El límite del Call Stack)
  - En la **versión recursiva**, si intentas ordenar una lista masiva, las llamadas acumuladas agotarán esos pocos megabytes y el programa se cerrará abruptamente (crash).
  - En la **versión iterativa**, tú creas la pila usando malloc(). Esto ubica tu pila en el **Heap** (memoria dinámica), que tiene acceso a casi toda la memoria RAM de tu computadora (Gigabytes).
- El peligro del peor caso de Quick Sort (N^2):
  - Para N = 100,000 elementos, la recursividad intentaría abrir 100,000 funciones al mismo tiempo.
  - La versión iterativa simplemente guardará 200,000 números enteros en su arreglo interno

```
// Función auxiliar para imprimir
void imprimirArreglo(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    // Generar la semilla aleatoria usando la hora actual del sistema
    srand(time(NULL));

    int n = 8; // Cantidad de elementos
    int arr[n];

    // Llenar el arreglo con números aleatorios entre 1 y 99
    for(int i = 0; i < n; i++) {
        arr[i] = (rand() % 99) + 1;
    }

    printf("Arreglo original (aleatorio):\n");
    imprimirArreglo(arr, n);

    quickSortIterativo(arr, 0, n - 1);

    printf("Arreglo ordenado:\n");
    imprimirArreglo(arr, n);

    return 0;
}
```

Quicksort iterativo

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h> // Necesario para generar la semilla de números aleatorios

// Función auxiliar para intercambiar dos elementos
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// 1. Partición de Lomuto: Siempre toma el último elemento como pivote
int particionLomuto(int arr[], int low, int high) {
    int pivote = arr[high];
    int i = (low - 1); // Índice de los elementos más pequeños

    // Recorrer el subarreglo desde 'low' hasta 'high - 1'
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] <= pivote) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    // Ubicar el pivote en su posición final correcta
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}
```

```c
void quickSortIterativo(int arr[], int l, int h) {
    // Reservar memoria para la pila de índices
    int* pila = (int*)malloc((h - l + 1) * sizeof(int));
    int tope = -1;

    // Push inicial de los límites del arreglo completo
    pila[++tope] = l;
    pila[++tope] = h;

    // Ciclo principal mientras haya límites en la pila
    while (tope >= 0) {
        // Pop de los límites actuales
        h = pila[tope--];
        l = pila[tope--];

        // Particionar utilizando Lomuto
        int p = particionLomuto(arr, l, h);

        // Si hay un subarreglo izquierdo válido, hacer Push a la pila
        if (p - 1 > l) {
            pila[++tope] = l;
            pila[++tope] = p - 1;
        }

        // Si hay un subarreglo derecho válido, hacer Push a la pila
        if (p + 1 < h) {
            pila[++tope] = p + 1;
            pila[++tope] = h;
        }
    }
    free(pila); // Evitar fugas de memoria
}
```

Quicksort iterativo: ejercicios

1. Ejercicio de Escritorio (Papel y Lápiz):

- **Objetivo:** Comprender exactamente cómo los índices entran y salen del Stack *simulado*, consolidando el concepto LIFO.
- **Consigna:** Se da el arreglo `[8, 2, 5, 3, 9, 4]`. Asumiendo que usamos partición de Lomuto (el último elemento es el pivote), dibuja el estado del arreglo y el contenido exacto de la pila de índices `[low, high]` en cada iteración del ciclo `while` hasta que la pila quede vacía.

2. Ejercicio de Adaptación (Programación en C): "Quick Sort en el Catálogo"

- **Objetivo:** Aplicar el algoritmo genérico de enteros a un Tipo de Dato Abstracto (TAD) con estructuras reales.
- **Consigna:** Tomando el TAD de un `Catalogo` de películas, implementa la función `ordenarPorId_QuickSortIterativo(Catalogo cat)`.